
Consolidating Reasoning with Test-Time Learning

Haotian Wu* Zeming Chen* Hao Zhao Antoine Bosselut

EPFL, Switzerland

{haotian.wu, zeming.chen, antoine.bosselut}@epfl.ch

Abstract

Parallel thinking scales inference-time compute by generating multiple reasoning traces for a query and aggregating them to reach a final answer. However, most aggregation algorithms treat each reasoning trajectory independently, ignoring the complementary reasoning sub-processes across the full ensemble of trajectories.

To learn from these parallel trajectories, we propose **CORAL (Consolidating Reasoning with Test-Time Learning)**, a novel framework for aggregating trajectories by jointly consolidating them into parametric memory using test-time training. Our algorithm meta-learns *how to consolidate* using nested optimization at training time, where an inner loop encodes reasoning trajectories into a lightweight LoRA adapter, and an outer loop learns meta-parameters for the inner loop such that the consolidation is helpful for the model to synthesize a final solution.

As no standard datasets exist for training aggregation methods, we construct a novel *OpenParallelThinking* corpus composed of 12,810 math problems paired with a mixture of diverse trajectories from six LLMs. Evaluated on challenging mathematics and STEM benchmarks, CORAL consistently outperforms the best aggregation baselines by 20% overall. Importantly, we curate reasoning trajectories at multiple quality levels to simulate the reality of noisy parallel reasoning and demonstrate that CORAL is especially robust to the quality of thinking trajectories and able to learn useful knowledge from entirely wrong solutions. Additionally, CORAL can recover from unsuccessful trajectories and arrive at an improved solution through consolidation as test-time learning, an ability that scales with the number of incorrect trajectories available, while RL further improves the adapted policy’s ability to reason over the consolidated trajectories. We will release CORAL and *OpenParallelThinking* for use in future work on reasoning aggregation.

1 Introduction

Scaling inference-time compute improves reasoning in large language models (LLMs) [7, 33, 47]. A particularly promising strategy is parallel thinking, where multiple (potentially diverse) reasoning trajectories are generated in parallel and then aggregated to yield a final answer (e.g., most simply by majority voting). These trajectories constitute distinct computational paths, with different intermediate conclusions, problem decompositions, and activated knowledge, providing an array of generated information that models can exploit to reach better conclusions. The empirical payoff of parallel thinking has been impressive. Gemini Deep Think, for example, achieved an official gold-medal performance at IMO 2025 by exploring and combining multiple candidate hypotheses in parallel rather than committing to a single chain of thought [23].

However, common trajectory aggregation methods, such as majority voting, reward-guided selection, and best-of- N sampling [7, 58, 62] do not effectively leverage all available reasoning signals in

*Equal contribution.

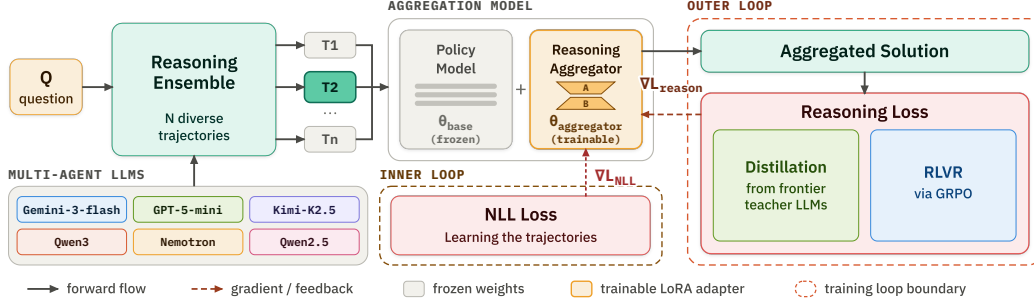


Figure 1: **Meta-learning CORAL for reasoning consolidation.** The training procedure involves a nested inner and outer loop. The inner loop optimizes the likelihood of a batch of reasoning trajectories with respect to the parameters of the LoRA reasoning aggregator. In the outer loop, the model uses the merged reasoning information in the aggregator to synthesize the final solution. In both loops, only the LoRA is updated while the base LLM is frozen.

these trajectories, often focusing only on the final answer reached by the trajectory. In effect, these methods treat each the *reasoning* in trajectory independently, aggregate final *answers* for a vote, and leave unexploited the potentially complementary reasoning encoded across the full ensemble of trajectories [21]. Although recent work proposes model-based summarization to better harness the reasoning across trajectories [10, 43, 69], these are attention-based aggregates, suffering from typical long-context degradation issues [17, 26, 28].

We propose **CORAL (Consolidating Reasoning with Test-Time Learning)**, a meta-learned framework that transforms trajectory aggregation from passive selection into active consolidation as test-time learning. At inference, CORAL consolidates diverse reasoning trajectories in a lightweight LoRA adapter via gradient updates and then synthesizes a final solution from the adapted model. We train the adapter via bi-level optimization: the inner loop minimizes a self-supervised next-token-prediction loss over a batch of trajectories, while the outer loop is trained to produce a synthesized answer, either through supervised finetuning on demonstrations distilled from a frontier teacher or through RLVR with GRPO [16, 45]. The outer-loop objective backpropagates through the inner loop to shape how the LoRA consolidates the ensemble end-to-end, teaching the adapter to consolidate trajectories into representations that improve reasoning.

We train CORAL on *OpenParallelThinking*, a novel dataset containing 13K math problems, each paired with groups of reasoning trajectories from 6 LLMs, which we will release for training reasoning aggregators at scale. Across math and STEM benchmarks, CORAL outperforms all rule- and model-based aggregation methods, with average gains of up to 20% over the strongest baselines. Importantly, CORAL actually learns from the reasoning behavior in the trajectories, rather than shortcutting to voting based on the final answers in trajectories. Consequently, on weak mixtures where no input trajectory is correct, CORAL can recover from these unsuccessful trajectories and arrive at an improved solution, outperforming the best baseline by up to 23%, and its gains scale with the number of weak trajectories in consolidation. Finally, RL further improves the adapted policy’s ability to reason over consolidated trajectories, adding an average 5-point gain.

2 Background and Preliminaries

2.1 Test-Time Scaling and Reasoning Aggregation

Scaling inference-time compute complements scaling parameters or pre-training data for improving LLM reasoning [62]. Among test-time scaling strategies, *parallel thinking*, i.e., sampling multiple reasoning trajectories and aggregating them into a single answer, has yielded remarkable empirical gains [23]. Formally, given a question Q with verifiable ground-truth answer y^* , parallel thinking involves sampling N reasoning trajectories $\mathcal{R} = \{R_1, \dots, R_N\}$ from a causal language model p_θ parameterized by θ , where each R_n is a complete chain of thought ending in a candidate answer a_n . An *aggregator* maps the question and trajectories to a final prediction. We write this as a generic operator: $\hat{y} = \mathcal{A}(Q, \mathcal{R}; \theta)$. The design space of $\mathcal{A}$ has evolved through three progressively more expressive paradigms.

Rule-Based Aggregation. Standard aggregations apply hand-crafted rules to conclude a final answer from $\mathcal{R}$. *Majority Voting* [58] returns the modal answer, while *reward-guided selection* [62] ranks trajectories with an external scorer r and keeps the top one:

$$\hat{y}_{\text{rule}} = \begin{cases} \text{mode}(\{a_n\}_{n=1}^N) & \text{(majority vote)} \\ a_{n^*}, \quad n^* = \arg \max_{n \in [N]} r(Q, R_n) & \text{(reward selection)} \end{cases} \quad (1)$$

Rule-based aggregation discards intermediate reasoning, so partial correctness in trajectories with wrong final answers (e.g., a correct sub-derivation or a pruned but valid case) is never leveraged.

Model-Based In-Context Aggregation. To exploit intermediate reasoning, recent work concatenates the trajectories into the prompt and produces an aggregated answer through a single forward pass with *frozen* parameters [53, 55]: $\hat{y}_{\text{ctx}} \sim p_{\theta}(\cdot \mid Q \cdot R_1 \cdot R_2 \cdots R_N)$. Information from $\mathcal{R}$ enters the model only through self-attention over the context. As N grows, the context length scales linearly, and LLMs are well documented to degrade on long, distractor-rich inputs [17, 26, 28, 30, 64], leaving the rich complementary reasoning in $\mathcal{R}$ underexploited.

RL-Trained Model Aggregation. A natural refinement keeps the in-context interface but *trains* the aggregator end-to-end with reinforcement learning from verifiable rewards (RLVR), maximizing the expected reward of the aggregated output [53, 69]:

$$\theta^{\text{RL}} = \arg \max_{\theta} \mathbb{E}_{(Q, \mathcal{R}, y^*)} \left[\mathbb{E}_{\hat{y} \sim p_{\theta}(\cdot \mid Q, \mathcal{R})} [r(\hat{y}, y^*)] \right]. \quad (2)$$

This turns aggregation into a learned skill rather than a fixed heuristic, but still consumes $\mathcal{R}$ only through attention over a concatenated context, where the same long-context bottleneck persists.

2.2 Test-Time Training

A common thread in prior paradigms is that θ is fixed at inference, and trajectories are processed only through context. In this work, we propose to perform aggregation with *Test-time training* (TTT), consolidating the trajectories $\mathcal{R}$ directly in the model’s parameters via S gradient steps using a self-supervised language modeling objective $\mathcal{L}_{\text{NLL}}$:

$$\phi^{(s)} = \phi^{(s-1)} - \alpha \nabla_{\phi^{(s-1)}} \mathcal{L}_{\text{NLL}}(\phi^{(s-1)}; \mathcal{R}), \quad s = 1, \dots, S, \quad (3)$$

starting from $\phi^{(0)} = \theta$ and yielding adapted parameters $\phi(\theta, \mathcal{R}) \triangleq \phi^{(S)}$. The aggregated final prediction is then drawn from the *updated* model: $\hat{y}_{\text{TTT}} \sim p_{\phi(\theta, \mathcal{R})}(\cdot \mid Q)$. This formulation is motivated by the observation that LLMs store and retrieve knowledge more reliably through their parameters than long-range attention [13, 49, 72], sidestepping the long-context bottleneck that limits model-based aggregation.

Meta-Learning for Test-Time Training. Training an LLM for TTT is naturally a meta-learning problem [12, 13, 19, 54], where initial parameters θ are meta-learned such that gradient-based adaptation of any $\mathcal{R}$ at test time yields strong downstream performance. With $\phi(\theta, \mathcal{R})$ denoting the post-adaptation parameters, the meta-objective is the bilevel optimization

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(Q, \mathcal{R}, y^*)} \left[\mathcal{L}_{\text{inner}}(\phi(\theta, \mathcal{R}); Q, y^*) \right], \quad (4)$$

where $\mathcal{L}_{\text{inner}}$ governs how $\mathcal{R}$ is internalized and $\mathcal{L}_{\text{outer}}$ scores the adapted model on the downstream task. Following common practice [13, 31, 49], $\mathcal{L}_{\text{inner}}$ is the negative log-likelihood (NLL) over $\mathcal{R}$,

$$\mathcal{L}_{\text{inner}}(\phi; \mathcal{R}) = - \sum_{k=1}^N \sum_{t=1}^{|R_k|} \log p_{\phi}(R_{k,t} \mid R_{k,<t}), \quad (5)$$

which encodes the reasoning traces into the model’s parameters. The outer-loop loss admits two natural instantiations:

$$\mathcal{L}_{\text{outer}}(\phi; Q, y^*) = \begin{cases} - \sum_{t=1}^{T_{y^*}} \log p_{\phi}(y_t^* \mid Q, y_{<t}^*) & \text{(SFT)} \\ - \mathbb{E}_{y \sim p_{\phi}(\cdot \mid Q)} [r(y, y^*)] & \text{(RLVR)} \end{cases} \quad (6)$$

The SFT objective provides a dense, low-variance signal but requires annotated demonstrations, while the RLVR objective directly optimizes the verifiable performance at the cost of higher variance. CORAL builds on this formulation by instantiating the inner loop as a parameter-efficient (LoRA) adaptation that consolidates $\mathcal{R}$, and the outer loop as a learned synthesis of a final chain-of-thought (CoT) solution from the adapted model.

3 CORAL: Reasoning Consolidation as Test-Time Learning

We instantiate the **Test-Time Training** (TTT) framework from Section 2 for reasoning aggregation. Rather than treating $\mathcal{R}$ as a passive context, CORAL consolidates the trajectories into LoRA adapter weights ϕ [27] attached to a frozen backbone θ_{base} via gradient-based inner-loop adaptation, then answers Q using the adapted model. Figure 1 shows the procedure.

Inner Loop Consolidation. For each training example $(Q, \mathcal{R}, y^*)$, the inner loop initializes $\phi^{(0)} = \theta_{\text{adapter}}$ and adapts only ϕ while keeping θ_{base} fixed:

$$\phi^{(s)} = \phi^{(s-1)} - \alpha \nabla_{\phi} \mathcal{L}_{\text{inner}}(\phi^{(s-1)}; Q, \mathcal{R}), \quad s = 1, \dots, S, \quad (7)$$

where the consolidation loss is the question-conditioned NLL of the sampled reasoning trajectories,

$$\mathcal{L}_{\text{inner}}(\phi; Q, \mathcal{R}) = - \sum_{k=1}^K \sum_{t=1}^{|\mathcal{R}_k|} \log p_{\theta_{\text{base}} \oplus \phi}(R_{k,t} \mid Q, R_{k,<t}). \quad (8)$$

Thus, the adapted $\phi^{(S)}$ serves as a parametric consolidation of $\mathcal{R}$. At inference time, the final answer is generated from Q alone using $p_{\theta_{\text{base}} \oplus \phi^{(S)}}(\cdot \mid Q)$.

Outer Loop Meta-Learning The outer loop optimizes θ_{adapter} by differentiating the meta objective through the inner-loop updates. For SFT, we use the first branch of Eq. (6) with p_{ϕ} replaced by $p_{\theta_{\text{base}} \oplus \phi^{(S)}}$. The adapted model is trained using solution demonstrations from frontier teacher models, GPT-5.4-xhigh and Gemini 3.1 Pro Deep Think, which provide high-quality CoT solutions as supervised targets [40, 50, 51].

For RLVR, we use the GRPO loss over the adapted policy (base model with adapted LoRA):

$$\begin{aligned} \mathcal{L}_{\text{GRPO-MAML}} &= -\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t} \hat{A}_i, \text{clip}(r_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right), \\ r_{i,t} &= \frac{p_{\theta_{\text{base}} \oplus \phi^{(S)}}(y_{i,t} \mid Q, y_{i,<t})}{p_{\theta_{\text{base}} \oplus \phi_{\text{old}}^{(S)}}(y_{i,t} \mid Q, y_{i,<t})}. \end{aligned} \quad (9)$$

Here $\{y_i\}_{i=1}^G$ are sampled responses from the adapted policy, $\hat{A}_i$ are group-normalized advantages, and $\phi_{\text{old}}^{(S)}$ is the frozen reference copy of the adapted LoRA weights. To reduce memory, we apply truncated gradient unrolling (Appendix D).

3.1 The *OpenParallelThinking* Dataset

Existing reasoning datasets typically pair each problem with one solution, leaving little supervision for consolidating multiple reasoning traces. We address this gap with *OpenParallelThinking*, a dataset for training reasoning aggregators.

OpenParallelThinking targets three design goals: source diversity, trajectory diversity, and balanced difficulty. We pool problems from four established math corpora and deduplicate them, then generate trajectories from four solvers spanning different scales, families, and training recipes, so the aggregator is not biased toward a single reasoning style. Trajectories are grouped per question under a token budget, and groups are filtered by correctness ratio to emphasize aggregation over predominantly incorrect trajectories, which yields a stronger learning signal than relying on a majority-correct shortcut (Figure 2). Appendix C.2 gives the full construction pipeline, including source datasets, solver selection, deduplication, grouping, and filtering. The final release contains 12,810 questions and 63,507 trajectory groups, drawn from 19,615 unique deduplicated problems. We sample 8,000 instances for RL training and reserve the rest for SFT. We will release the dataset and construction pipeline to support future work on reasoning aggregation.

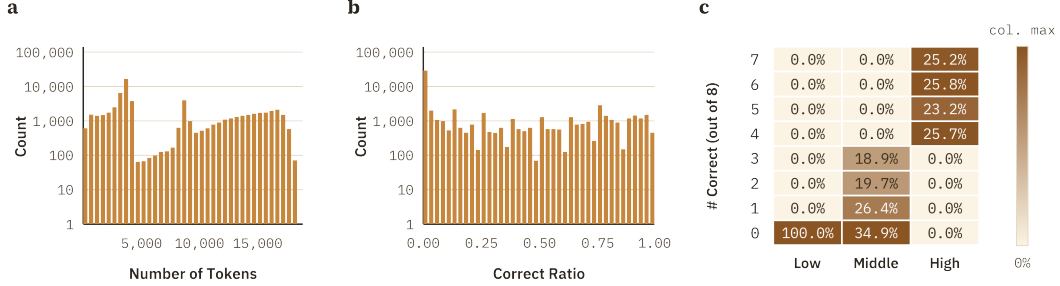


Figure 2: **Left:** Token length distribution of the *OpenParallelThinking*. **Center:** Correct ratio distribution *OpenParallelThinking*, reflecting a cosine-like difficulty profile shaped by correctness-based filtering. **Right:** Correctness distribution of the evaluation trajectory groups across three tiers.

4 Experiments

4.1 Experimental Setup

Benchmarks for Reasoning Aggregation. We evaluate on reasoning-heavy, meaningful benchmarks across two domains. For mathematical reasoning, we use AIME2025 [2], AIME2026 [3], Apex2025, ArxivMath, CMIMC2025, HMMT-feb-2026, and HMMT-nov-2025 from MathArena [4], and IMO-Bench [39]. For generalization beyond math, we use two general STEM benchmarks: GPQA-diamond [44] and TheoremQA [9]. For every problem, we generate eight reasoning trajectories (each ends with a final answer) distilled from LLMs spanning a range of capability levels. The generation and filtering are detailed in Appendix C.1. We group these trajectories into a quality bucket that isolates the effect of trajectory quality on aggregation performance: *Minority* (0–3 correct). Essentially, having ≥ 4 correct trajectories already allows a simple rule-based aggregation like Majority Voting to get 100% accuracy, which is unlikely to occur naturally. We additionally group trajectories into *None*, where all trajectories are incorrect, to analyze whether CORAL can learn from their partial reasoning and aggregate them into a more reliable solution.

Baselines. We compare our method with four reasoning aggregation methods. For rule-based aggregation, we evaluate majority voting [58]. For model-based aggregation, we compare with three variants: prompt aggregation, SFT aggregation, and RL aggregation [69]. All model-based methods perform in-context aggregation at inference. Prompt aggregation uses the base model directly, whereas SFT and RL aggregation use models trained with the corresponding objectives. We use Qwen3-1.7B and Qwen3-4B [65] as the base models, and for all small datasets (less than 100 samples), and run them 8 times individually and report their average scores. For non-aggregation methods, we compare with the standard reasoning-heavy R1-style inference [24] with a 16K token extended thinking budget. See Appendix D, E for other implementation details.

4.2 Main Results

CORAL consistently outperforms baseline aggregation methods. Across ten benchmarks and both trajectory-quality tiers, CORAL outperforms the baseline aggregation methods in nearly every case (Table 1). At fixed model scale, the gain is stronger in the more difficult *None* tier, where CORAL’s advantage grows precisely where input trajectories all contain incorrect conclusions and rule-based aggregation fails. At fixed tier, scaling from 1.7B to 4B roughly doubles the gain, indicating that a stronger aggregator extracts more signal from the same set of trajectories. Compared to extended thinking with a 16K-token budget, CORAL trails by only 4.9 (1.7B) and 4.5 (4B) points when only aggregating incorrect trajectories (*None*). With *Minority*, CORAL surpasses extended thinking by 7 (1.7B) and 10 (4B) points, with the gap widening at a larger scale. In contrast, every baseline aggregator trails extended thinking by a large margin, even in the *Minority* setting. Notably, SFT Aggregation is trained on the same questions, trajectory groups, and frontier-generated target solutions as CORAL, so this gap cannot be explained by frontier supervision in the outer loop alone: CORAL’s consolidation mechanism is what drives the improvement.

Table 1: **Main results (%) on math and STEM reasoning benchmarks.** Each row group corresponds to a model scale and trajectory quality setting. *None*: All 8 trajectories are incorrect. *Minority*: A minority of trajectories are correct. **Bold** shows the best result per column within each group. Underline reflects the best performance across all groups.

		Math Reasoning								STEM	
		AIME25	AIME26	Apex25	ArXivMath	CMIMC25	HMMT-2602	HMMT-2511	IMO-Bench		
Setting	Method									GPQA	TheoremQA
Extended Thinking	Qwen3-1.7B	40.0	40.8	0.0	13.9	25.3	21.9	25.8	22.8	37.4	57.5
	Qwen3-4B	<u>71.7</u>	<u>68.3</u>	0.0	16.7	52.2	40.5	60.4	37.0	51.5	<u>68.6</u>
Rule-based Agg.	Majority Voting @8 (None)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	Majority Voting @8 (Minority)	26.7	23.3	25.0	18.1	30.0	30.3	16.7	21.5	16.2	19.5
Qwen3-1.7B None	Prompt Aggregation (CoT)	5.0	6.7	0.0	0.0	0.0	2.7	2.1	1.5	7.1	14.6
	Prompt Aggregation (Thinking)	22.9	21.7	0.0	2.8	10.9	17.8	7.9	7.5	13.6	25.1
	SFT Aggregation (CoT)	2.5	5.4	0.0	1.4	2.5	4.2	3.3	0.8	0.5	11.1
	RL Aggregation (Thinking)	2.9	6.7	0.0	2.8	0.3	0.4	2.9	2.0	7.1	13.3
	CORAL[†]	36.6	26.7	16.7	23.6	20.0	18.2	23.3	14.5	22.7	33.5
Qwen3-1.7B Minority	Prompt Aggregation (CoT)	23.3	15.8	0.0	13.9	13.8	15.2	20.0	15.0	28.3	26.4
	Prompt Aggregation (Thinking)	36.3	42.1	3.1	16.7	28.1	25.8	21.7	18.5	28.8	37.3
	SFT Aggregation (CoT)	13.3	14.6	0.0	4.2	12.5	12.1	4.6	7.8	15.2	18.1
	RL Aggregation (Thinking)	31.3	25.4	0.0	19.5	23.8	22.4	23.8	17.3	28.8	27.3
	Recursive Refinement	26.7	18.9	0.0	11.1	21.7	18.2	14.4	23.5	26.8	—
	CORAL[†]	43.3	30.0	25.0	34.7	45.0	27.3	36.4	25.5	42.9	43.6
Qwen3-4B None	Prompt Aggregation (CoT)	11.7	4.6	0.0	4.2	4.4	9.1	6.7	3.0	10.1	21.3
	Prompt Aggregation (Thinking)	32.9	36.7	0.0	5.6	24.5	24.6	14.2	12.0	17.2	28.6
	SFT Aggregation (CoT)	16.7	17.9	0.0	4.2	3.4	4.9	6.7	3.3	2.0	17.6
	RL Aggregation (Thinking)	15.8	15.8	16.7	5.6	15.0	9.1	20.0	6.7	4.3	32.6
	CORAL[†]	69.5	39.1	33.3	37.9	35.0	48.5	52.3	28.3	22.5	41.9
Qwen3-4B Minority	Prompt Aggregation (CoT)	28.3	26.7	0.0	16.7	31.9	25.0	26.7	18.0	29.3	33.4
	Prompt Aggregation (Thinking)	61.7	61.7	2.1	23.6	44.1	26.1	35.8	23.8	30.8	39.8
	SFT Aggregation (CoT)	30.0	23.3	8.3	6.9	15.0	18.2	20.0	7.0	3.5	22.0
	RL Aggregation (Thinking)	56.7	53.3	8.3	23.6	40.0	28.0	38.4	22.3	21.7	37.6
	CORAL[†]	69.6	47.6	41.6	42.9	78.6	57.7	72.7	38.3	64.6	61.9

CORAL can aggregate unsuccessful trajectories into a more reliable solution. In the *None* setting, every input trajectory arrives at an incorrect final solution, forcing majority voting to 0% performance. Averaged across all ten benchmarks, CORAL achieves absolute gains of 22.6 and 10.6 points, approximately doubling the the strongest baseline performance (Figure 3). The results suggest that test-time learning to consolidate can encode useful partial reasoning from collectively wrong attempts, producing answers stronger than individual attempts or the aggregation model solving the problem alone ².

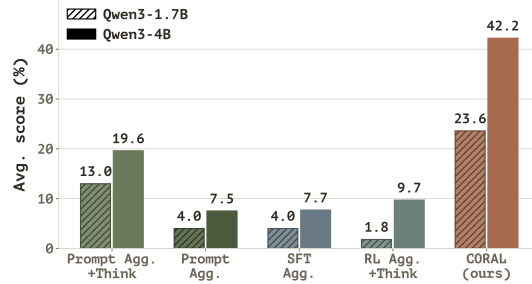


Figure 3: **Aggregate collectively wrong trajectories *None* trajectories with CORAL.** CORAL improves the most compared to other methods.

CORAL demonstrates out-of-domain transfer. CORAL is trained only on math reasoning from *OpenParallelThinking* we constructed, but its advantage generalizes cleanly to scientific domains. Across GPQA and TheoremQA, CORAL averages 14 points above the strongest baseline across trajectory quality tiers and model scales, comparable in magnitude to its in-domain mathematical gains, suggesting consolidation based on test-time learning is a domain-general reasoning operation.

5 Analysis

5.1 CORAL’s performance scales as the number of weak trajectories increases

²We further show qualitatively that CORAL can consolidate failed attempts into an improved solution. Three frontier LLM judges (Table 5, Opus 5, and GPT-5.6-Sol; extra-high) compare 100 CORAL solutions on the *None* tier with their input trajectories. 98% show clear synthesis of a better reasoning trajectory from the incorrect inputs. An audit of a separate 100 *Minority*-tier solutions finds only 1 case of copying the input trajectory. Appendix C.4 shows the judge protocol and details.

Table 3: **Accuracy vs. compute**, *None* tier, averaged across the three benchmarks in Figure 3.

K	MV FLOPs	MV	In-context FLOPs	Prompt Agg.	SFT	RL Agg.	CORAL FLOPs	CORAL
2	1.755e14	0.0	2.227e14	4.3	2.2	1.9	2.897e14	15.2
4	3.509e14	0.0	4.105e14	3.4	3.9	1.7	5.432e14	23.0
8	7.019e14	0.0	7.890e14	5.5	3.7	2.4	1.050e15	23.6
16	1.404e15	0.0	1.558e15	4.7	6.7	0.8	2.064e15	26.5

We analyze how aggregation methods scale with the number of weak trajectories under the *None* tier. With all trajectories being incorrect, we rule out the shortcut of selecting an already-correct candidate, directly testing whether each method can extract useful knowledge from wrong attempts. We grow the number of trajectories as $n \in \{2, 4, 8, 16\}$, with Qwen3-1.7B as the backbone.

We show in Figure 4 that as more incorrect trajectories are added, the baseline performance stagnates or even degrades.

In contrast, CORAL generally benefits from more trajectories during consolidation, for instance, 11.1% increase on GPQA. We hypothesize that this stems from test-time learning deriving informative signal from noisy additional trajectories.

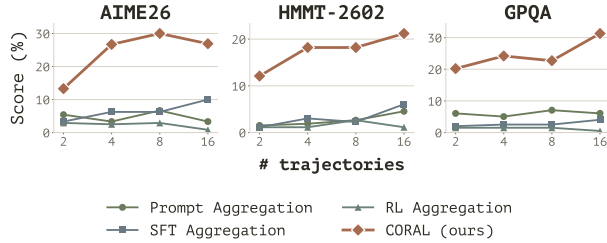


Figure 4: **Scaling behavior of aggregation methods with the # of weak trajectories.** As more *None*-quality trajectories (carry incorrect answers) are added, baselines improve slightly or degrade while CORAL continues to improve.

5.2 CORAL outperforms compute-matched baselines

CORAL outperforms majority voting at matched compute. We verify if CORAL’s gain comes from the extra test-time compute by checking if a compute-matched baseline could achieve a similar performance. We allocate CORAL’s TTL budget to additional majority-voted rollouts (Table 2). Even with $1.50\times$ more FLOPs, majority voting @18 is still worse by 12.8 points than CORAL@8, and it does not meaningfully improve as the compute grows ($22.7 \rightarrow 22.2 \rightarrow 22.6$). Appendix A extends this comparison to reward-guided and recursive-refinement baselines on latency and memory.

Table 2: **Matched compute vs. majority voting (MV)**, *Minority* tier, CORAL Qwen3-1.7B.

Method	FLOPs	Relative Compute	Minority Acc.
MV @8	7.019e14	0.67×	22.7
MV @13	1.141e15	1.09×	22.2
MV @18	1.579e15	1.50×	22.6
CORAL[†] @8	1.050e15	1.00×	35.4

CORAL outperforms in-context aggregation even when using less compute. We further evaluate accuracy as a function of total computation, sweeping the number of incorrect trajectories K (*None*). We report average accuracy across the three benchmarks in Figure 4 with the FLOPs for trajectory sampling and reasoning aggregation. CORAL consistently outperforms every aggregation baseline even when they use $1.5\times$ more FLOPs and twice as many trajectories. CORAL’s performance increases with compute, from 15.2 to 26.5 when K grows to 16. Majority voting always scores 0.0 since all candidates are wrong. Equal or greater FLOPs on additional sampling or in-context aggregation does not match CORAL’s performance. CORAL’s gains arise from consolidating useful intermediate reasoning through test-time learning, not simply from more test-time computation.

5.3 CORAL requires joint consolidation in parameter space

We validate CORAL’s gains require cross-trajectory interaction, not repeated exposure to the same question or task-distribution activation. We compare to **Single-Trace $\times$ 8**: 8 separately adapted LoRAs, one per input trajectory, with a majority vote over their 8 generated solutions (Figure 5). Both see the same 8 trajectories and spend matched adaptation compute. The only difference is if cross-trajectory interaction happens before producing a solution. Single-Trace $\times$ 8 is worse by 10.3 points at matched compute, showing that CORAL’s gain requires consolidating trajectories *jointly* within a shared parameter space (adapted LoRA) rather than encoding them independently and reconciling only by voting over outputs. Appendix B.2 extends this comparison with two more controls, passive question

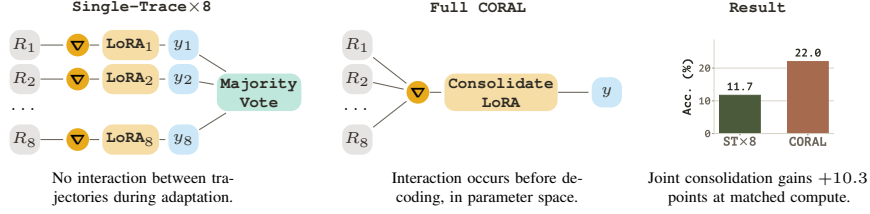


Figure 5: **Where trajectory information can interact.** Under Single-Trace $\times 8$ (left), each trajectory is adapted and decoded independently, then reconciled by a vote. Under full CORAL (middle), all 8 trajectories jointly update one Consolidate LoRA before any answer is generated. Single-Trace $\times 8$ underperforms full CORAL on the *None* tier (right).

repetition and task-distribution activation without target-specific trajectories, both of which perform worse than Single-Trace $\times 8$, ruling them out as alternative explanations.

5.4 CORAL consolidates reasoning rather than voting on the answers

An aggregation method that scores well when most trajectories carry the correct answer could either be consolidating reasoning or simply voting on final answers. To distinguish these, we use the *Majority* quality bucket: 4 to 7 trajectories carry correct answers. We swap the correct/incorrect answers so the majority answer becomes wrong, while reasoning stays untouched (*Majority-reversed*). A consolidator should be largely invariant, but a “voter” should collapse. We show the results in Figure 6.

Under the *Majority* tier, Prompt and RL Aggregation reach 80–90% on several benchmarks. However, after reversal, RL drops by up to 64 points and Prompt by up to 52, often falling below 30%. The large drop suggests their high scores came from shortcut voting on final answers rather than understanding the trajectories. In contrast, CORAL’s drop stays within the $[-17, -5]$ range across all benchmarks, indicating it utilizes information from trajectories rather than counting answers.

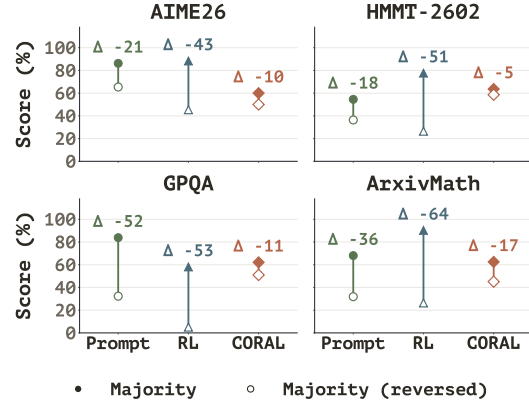


Figure 6: **Robustness to majority manipulation.** Dumbbells connect each method’s score under *Majority* (filled) and *Majority-reversed* (open, correct/incorrect answers swapped); bar length is the resulting drop. Baselines collapse while CORAL stays short in every panel, indicating consolidation rather than answer-tallying.

5.5 RL incentivizes CORAL’s reasoning over consolidated trajectories

Reinforcement learning with verifiable rewards (RLVR) has become a standard recipe for eliciting stronger reasoning from LLMs [24]. A natural question is whether such reward-driven optimization can also improve *reasoning consolidation*. We cast RLVR as the outer-loop task for meta-learning over CORAL (Qwen3-1.7B) where the inner loop performs consolidation, and the outer loop uses GRPO to maximize the likelihood that the consolidated policy reaches a correct answer [45].

Figure 7 compares CORAL (RL-enhanced) against the supervised CORAL on aggregating *None* tier trajectories. RL training lifts the average accuracy by 4.6 absolute gain (a $\sim 22\%$ relative improvement), with the largest gains on GPQA-Diamond, nearly doubling from 22.7 to 40.0 (+17.3). RL improves the final adapted policy’s ability to

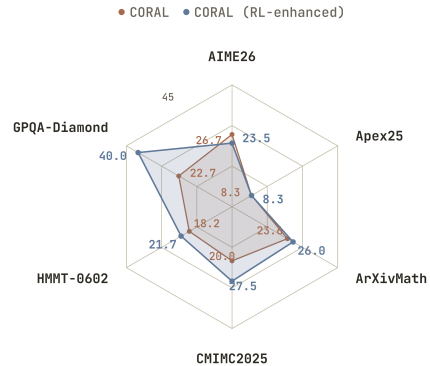


Figure 7: RL-enhanced CORAL with *None* tier trajectories, showing consistent gains over CORAL SFT.

reason over consolidated trajectories, while its current training gains arise mainly from a stronger initialization for consolidation.

6 Related Work

Reasoning Aggregation. A common way to combine multiple LLM solutions is rule-based voting: self-consistent decoding [7, 57] samples multiple reasoning trajectories and returns the most frequent answer, with later variants adapting the procedure through dynamic sampling or heuristic filtering [1, 8, 56]. This strategy breaks down when correct solutions occupy minority modes, causing voting to amplify errors rather than correct them. Learned models offer an alternative: reward models score and rank candidates [63, 68], Universal Self-Consistency [10] has the model inspect all sampled answers and select the most consistent one, and Zhao et al. [69] instead train the model to reason over and synthesize across candidates as an explicit skill. However, these methods all concatenate candidates directly into the context, suffering from attention dilution and rising cost as context grows. In contrast, CORAL meta-learns to proactively internalize cross-trajectory information rather than read over the candidate solutions.

Parallel Thinking. Parallel thinking has emerged as a promising paradigm for overcoming the fragility of single-trajectory sequential reasoning [25, 35, 59, 61, 67]. Most recent work trains models to natively generate and manage multiple thought branches concurrently: Parathinker [60] generates diverse, independent thoughts end-to-end and aggregates them into a final answer, while Parallel-R1 [70] first instills this ability via SFT on easy math problems (cold-start) before extending it to harder problems through reinforcement learning. Other approaches instead refine the structure of parallel reasoning branches [41, 66]; notably, Venkatraman et al. [53] frame the process as an evolutionary loop that recursively aggregates and refines populations of candidate solutions across iterations. Our method differs by decoupling generating parallel thoughts from reasoning over them, enabling the aggregation of thoughts from different sources beyond self-generated ones.

Test-Time Training. MAML [20] introduces optimization-based, model-agnostic meta-learning for efficient downstream adaptation; Chen et al. [12] apply MAML to GPT models, internalizing context knowledge via inner-loop gradient updates before answering queries in the outer loop. A related line of work instead adds or repurposes weight modules that mimic global sequence modeling while avoiding sequential dependency and quadratic scaling [15, 18], such as Sun et al.’s [48] RNN layers with MLP-based hidden states, and new architectures combining persistent neural memory with standard attention to extend context length while preserving context-grounding [5, 6]. Our method, CORAL apply test-time training by defining reasoning aggregation as a learning process.

7 Conclusion

We present CORAL, a meta-learned reasoning aggregation that actively consolidates thinking trajectories using test-time learning. Rather than reading trajectories as context, CORAL consolidates them in a lightweight LoRA adapter through a learned adaptation process, enabling the adapted LoRA to synthesize a more reliable solution. To enable training and evaluation at scale, we introduce *OpenParallelThinking*, a corpus of 13K problems paired with 63K trajectory groups spanning diverse models and three quality levels. Across ten challenging math and STEM benchmarks, CORAL outperforms the best aggregation baselines by 20% on average, with particularly notable gains when all input trajectories are incorrect, confirming that CORAL can recover from unsuccessful trajectories and arrive at an improved solution through aggregation as test-time learning. Further analysis shows that CORAL resists majority-voting shortcuts, scales positively with weak trajectory count, and that RL further improves the adapted policy’s ability to reason over consolidated trajectories.

Acknowledgments

We gratefully acknowledge the support of the Swiss National Science Foundation (No. 215390), the AI2050 program at Schmidt Sciences (Grant #G-25-69783), Sony Group Corporation, and the Swiss National Supercomputing Center (CSCS) in the form of an infrastructure engineering and development project. This research is with support from Google.org and the Google Cloud Research Credits program for the Gemini Academic Program. This project is also funded by the European Union (ERC, RESPECT-LM, 101222478). Views and opinions expressed are however those of the

author(s) only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- [1] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let’s sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12375–12396, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.761. URL <https://aclanthology.org/2023.emnlp-main.761/>.
- [2] AI-MO. Ai-mo/aimo-validation-aime. <https://huggingface.co/datasets/AI-MO/aimo-validation-aime>, 2025. 2025.
- [3] AI-MO. Ai-mo/aimo-validation-aime. <https://huggingface.co/datasets/AI-MO/aimo-validation-aime>, 2026. 2026.
- [4] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- [5] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [6] Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Atlas: Learning to optimally memorize the context at test time. *arXiv preprint arXiv:2505.23735*, 2025.
- [7] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- [8] Weizhe Chen, Sven Koenig, and Bistra Dilkina. Lspo: Length-aware dynamic sampling for policy optimization in llm reasoning. *arXiv preprint arXiv:2510.01459*, 2025.
- [9] Wenhui Chen, Ming Yin, Max Ku, Pan Lu, Yixin Wan, Xueguang Ma, Jianyu Xu, Xinyi Wang, and Tony Xia. Theoremqa: A theorem-driven question answering dataset. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7889–7901, 2023.
- [10] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311*, 2023.
- [11] Yang Chen, Zhuolin Yang, Zihan Liu, Chankyu Lee, Peng Xu, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Acereason-nemotron: Advancing math and code reasoning through reinforcement learning. *arXiv preprint arXiv:2505.16400*, 2025.
- [12] Zeming Chen, Gail Weiss, Eric Mitchell, Asli Celikyilmaz, and Antoine Bosselut. Reckoning: reasoning through dynamic knowledge encoding. *Advances in Neural Information Processing Systems*, 36:62579–62600, 2023.
- [13] Zeming Chen, Angelika Romanou, Gail Weiss, and Antoine Bosselut. PERK: Long-context reasoning as parameter-efficient test-time learning. *arXiv preprint arXiv:2507.06415*, 2025. URL <https://arxiv.org/abs/2507.06415>.
- [14] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

- [15] Karan Dalal, Daniel Kocejka, Jiarui Xu, Yue Zhao, Shihao Han, Ka Chun Cheung, Jan Kautz, Yejin Choi, Yu Sun, and Xiaolong Wang. One-minute video generation with test-time training. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17702–17711, 2025.
- [16] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [17] Shihan Dou, Ming Zhang, Zhangyue Yin, Chenhao Huang, Yujiong Shen, Junzhe Wang, Jiayi Chen, Yuchen Ni, Junjie Ye, Cheng Zhang, Huaibing Xie, Jianglu Hu, Shaolei Wang, Weichao Wang, Yanling Xiao, Yiting Liu, Zenan Xu, Zhen Guo, Pluto Zhou, Tao Gui, Zuxuan Wu, Xipeng Qiu, Qi Zhang, Xuanjing Huang, Yu-Gang Jiang, Di Wang, and Shunyu Yao. CL-bench: A benchmark for context learning. *arXiv preprint arXiv:2602.03587*, 2026. URL <https://arxiv.org/abs/2602.03587>.
- [18] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training. *arXiv preprint arXiv:2604.06169*, 2026.
- [19] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- [20] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- [21] Yichao Fu, Xuwei Wang, Yuandong Tian, and Jiawei Zhao. Deep think with confidence. *arXiv preprint arXiv:2508.15260*, 2025.
- [22] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. Omni-math: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*, 2024.
- [23] Google DeepMind. Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the international mathematical olympiad. <https://deepmind.google/blog>, July 2025. Accessed: 2026.
- [24] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shitong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [25] Qi Guo, Jianing Wang, Deyang Kong, Xiangyu Xi, Jianfei Zhang, Yi Lu, Jingang Wang, Wei Wang, Shikun Zhang, and Wei Ye. Ope: Overcoming information saturation in parallel thinking via outline-guided path exploration. *arXiv preprint arXiv:2602.08344*, 2026.
- [26] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesch, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.
- [27] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [28] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. *Advances in Neural Information Processing Systems*, 37:106519–106554, 2024.
- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

- [30] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353, 2024.
- [31] Gaotang Li, Ruizhong Qiu, Xiusi Chen, Heng Ji, and Hanghang Tong. Beyond log likelihood: Probability-based objectives for supervised fine-tuning across the model capability continuum. *arXiv preprint arXiv:2510.00526*, 2025.
- [32] Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. [<https://huggingface.co/AI-MO/NuminaMath-CoT>] (https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.
- [33] Zhenwen Liang, Ye Liu, Tong Niu, Xiangliang Zhang, Yingbo Zhou, and Semih Yavuz. Improving llm reasoning through scaling inference computation with collaborative verification. *arXiv preprint arXiv:2410.05318*, 2024.
- [34] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The twelfth international conference on learning representations*, 2023.
- [35] Houjun Liu, Shikhar Murty, Christopher D Manning, and Róbert Csordás. Thoughtbubbles: an unsupervised method for parallel thinking in latent space. *arXiv preprint arXiv:2510.00219*, 2025.
- [36] Zihan Liu, Yang Chen, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. AceMath: Advancing frontier math reasoning with post-training and reward modeling. *arXiv preprint arXiv:2412.15084*, 2024.
- [37] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [38] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL>, 2025. Notion Blog.
- [39] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- [40] OpenAI. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>, March 2026. Accessed: 2026-05-07.
- [41] Jiayi Pan, Xiuyu Li, Long Lian, Charlie Snell, Yifei Zhou, Adam Yala, Trevor Darrell, Kurt Keutzer, and Alane Suhr. Learning adaptive parallel reasoning with language models. *arXiv preprint arXiv:2504.15466*, 2025.
- [42] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37: 126544–126565, 2024.
- [43] Jianing Qi, Xi Ye, Hao Tang, Zhigang Zhu, and Eunsol Choi. Learning to reason across parallel samples for llm reasoning. *arXiv preprint arXiv:2506.09014*, 2025.

- [44] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.
- [45] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [46] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [47] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [48] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- [49] Arnub Tandon, Karan Dalal, Xinhao Li, Daniel Kocaja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, et al. End-to-end test-time training for long context. *arXiv preprint arXiv:2512.23675*, 2025.
- [50] The Deep Think Team. Gemini 3 Deep Think: Advancing science, research and engineering. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-deep-think/>, February 2026. Accessed: 2026-05-07.
- [51] The Gemini Team. Gemini 3.1 Pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>, February 2026. Accessed: 2026-05-07.
- [52] Shubham Toshniwal, Wei Du, Ivan Moshkov, Branislav Kisacanin, Alexan Ayrapetyan, and Igor Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. *arXiv preprint arXiv:2410.01560*, 2024.
- [53] Siddarth Venkatraman, Vineet Jain, Sarthak Mittal, Vedant Shah, Johan Obando-Ceron, Yoshua Bengio, Brian R Bartoldson, Bhavya Kailkhura, Guillaume Lajoie, Glen Berseth, et al. Recursive self-aggregation unlocks deep thinking in large language models. *arXiv preprint arXiv:2509.26626*, 2025.
- [54] Anna Vettoruzzo, Mohamed-Rafik Bouguelia, Joaquin Vanschoren, Thorsteinn Rögnvaldsson, and KC Santosh. Advances and challenges in meta-learning: A technical review. *IEEE transactions on pattern analysis and machine intelligence*, 46(7):4763–4779, 2024.
- [55] Fanqi Wan, Weizhou Shen, Shengyi Liao, Yingcheng Shi, Chenliang Li, Ziyi Yang, Ji Zhang, Fei Huang, Jingren Zhou, and Ming Yan. Qwenlong-11: Towards long-context large reasoning models with reinforcement learning. *arXiv preprint arXiv:2505.17667*, 2025.
- [56] Haonan Wang, Chao Du, Kenji Kawaguchi, and Tianyu Pang. Think in parallel, answer as one: Logit averaging for open-ended reasoning. *arXiv preprint arXiv:2512.02874*, 2025.
- [57] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [58] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [59] Ziqi Wang, Boye Niu, Zipeng Gao, Zhi Zheng, Tong Xu, Linghui Meng, Zhongli Li, Jing Liu, Yilong Chen, Chen Zhu, et al. A survey on parallel reasoning. *arXiv preprint arXiv:2510.12164*, 2025.

- [60] Hao Wen, Yifan Su, Feifei Zhang, Yunxin Liu, Yunhao Liu, Ya-Qin Zhang, and Yuanchun Li. Parathinker: Native parallel thinking as a new paradigm to scale llm test-time compute. *arXiv preprint arXiv:2509.04475*, 2025.
- [61] Haotian Wu, Bo Xu, Yao Shu, Menglin Yang, and Chengwei Qin. Thinking with nothinking calibration: A new in-context learning paradigm in reasoning large language models. *arXiv preprint arXiv:2508.03363*, 2025.
- [62] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2408.00724>.
- [63] Ran Xu, Yuchen Zhuang, Zihan Dong, Jonathan Wang, Yue Yu, Joyce C Ho, Linjun Zhang, Haoyu Wang, Wenqi Shi, and Carl Yang. Acesearcher: Bootstrapping reasoning and search for llms via reinforced self-play. *arXiv preprint arXiv:2509.24193*, 2025.
- [64] Xiaoyue Xu, Qinyuan Ye, and Xiang Ren. Stress-testing long-context language models with lifelong icl and task haystack. *Advances in Neural Information Processing Systems*, 37:15801–15840, 2024.
- [65] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [66] Xinyu Yang, Yuwei An, Hongyi Liu, Tianqi Chen, and Beidi Chen. Multiverse: Your language models secretly decide how to parallelize and merge generation. *arXiv preprint arXiv:2506.09991*, 2025.
- [67] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [68] Yue Yu, Zhengxing Chen, Aston Zhang, Liang Tan, Chenguang Zhu, Richard Yuanzhe Pang, Yundi Qian, Xuwei Wang, Suchin Gururangan, Chao Zhang, Melanie Kambadur, Dhruv Mahajan, and Rui Hou. Self-generated critiques boost reward modeling for language models. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 11499–11514, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.573. URL <https://aclanthology.org/2025.naacl-long.573/>.
- [69] Wenting Zhao, Pranjal Aggarwal, Swarnadeep Saha, Asli Celikyilmaz, Jason Weston, and Ilia Kulikov. The majority is not always right: RL training for solution aggregation. *arXiv preprint arXiv:2509.06870*, 2025.
- [70] Tong Zheng, Hongming Zhang, Wenhao Yu, Xiaoyang Wang, Runpeng Dai, Rui Liu, Huiwen Bao, Chengsong Huang, Heng Huang, and Dong Yu. Parallel-rl: Towards parallel thinking via reinforcement learning. *arXiv preprint arXiv:2509.07980*, 2025.
- [71] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <http://arxiv.org/abs/2403.13372>.
- [72] Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. *arXiv preprint arXiv:2506.10943*, 2025.

A Compute, Latency, and Efficiency Analysis

We verify that CORAL’s gains come from its consolidation mechanism rather than simply more test-time computation. We first give a theoretical complexity analysis of CORAL against majority voting and in-context aggregation, then present empirical evidence against compute-matched baselines.

A.1 Theoretical Complexity Analysis

Let K denote the number of intermediate trajectory samples, L the approximate number of tokens in each trajectory, and T the length of the final generated solution. We analyze scaling with K while holding the model, L , and T fixed. Under a token-proportional computation model, sampling K trajectories requires $\Theta(KL)$ total computation (Table 4). Majority voting therefore has a wall-clock advantage when parallel hardware is sufficient: all trajectories can be generated in parallel, while voting cost is negligible. However, this does not make additional trajectories free. Generating K trajectories concurrently still requires $\Theta(KL)$ total model computation. CORAL has the same linear asymptotic growth in total computation but a less favorable latency profile, because each consolidation update is sequentially dependent on the updated model state. Its advantage is accuracy per unit of total computation; we do not claim that CORAL dominates majority voting on every system dimension, but it is substantially more effective when total computation or serving cost is important, because equal or greater computation spent on additional rollouts does not recover its performance.

Table 4: Asymptotic scaling with the number of sampled trajectories K (fixed L, T).

Method	Total compute as K increases	Latency, sufficient parallel hardware
Majority voting	$\Theta(KL)$ sampling + $\Theta(K)$ voting = $\Theta(KL)$	$\Theta(L)$, independent of K with concurrent generation
CORAL	$\Theta(KL)$ sampling + $\Theta(KL)$ consolidation + $\Theta(T)$ generation = $\Theta(KL + T)$	Sampling parallelizes; consolidation is sequential, $\Theta(KL)$, followed by $\Theta(T)$
In-context aggregation	$\Theta(KL)$ sampling + $\Theta((KL)^2)$ prefill + $\Theta(TKL + T^2)$ decode = $\Theta(K^2)$ w.r.t. K	Sampling parallelizes; aggregation cost grows with context length

A.2 Empirical Evidence at Matched Compute

Section 5.2 reports CORAL’s accuracy against majority voting and in-context aggregation at matched compute (Tables 2 and 3). Here we extend that comparison to reward-guided and recursive-refinement baselines, and to inference latency and memory.

We also compare against three additional baselines at matched compute, using CORAL @8’s budget of 1.050e15 total FLOPs: reward-model Best-of-N [14] using AceMath-7B-RM [36], PRM reranking using Qwen2.5-Math-PRM-7B [34], and Recursive Self-Aggregation [53]. Recursive refinement receives $N=8$ candidate solutions and runs $T=2$ rounds; in each round, it draws $N=8$ subsets of $k=4$ candidates and generates one new candidate per subset, replacing the population. Table 5 shows that CORAL outperforms every baseline at equal compute, both for *None* and *Minority* tiers.

Latency and memory. We measure inference latency using Qwen3-1.7B on a single H100 GPU. All methods receive the same precomputed trajectories, so the latency for sampling trajectories is not counted as part of the total latency since all methods share this measurement. For CORAL, “Consolidate” denotes the complete test-time learning stage that consolidates all trajectories through LoRA updates. Table 6 shows that CORAL achieves the lowest latency compared to the aggregation baselines, being $2.18\times$ faster than the strongest baseline (25.238s vs. 55.038s for Prompt Aggregation in Thinking mode) while improving *Minority* accuracy by 9.6 points and *None* accuracy by 10.6 points. CORAL has higher peak GPU memory because test-time learning requires gradients and optimizer states. However, it uses a substantially smaller KV cache at decode time because the consolidated model generates from the 156-token problem prompt rather than attending to all concatenated trajectories; this also avoids the growth in KV memory incurred by in-context aggregation as the number of trajectories increases.

The gain comes from CORAL’s consolidation mechanism, not more test-time computation. Giving the baselines more compute still does not reproduce CORAL’s performance.

Table 5: Iso-compute comparison against reward-guided and recursive-refinement baselines.

Method	Setting	Pipeline FLOPs	Minority	None
Best-of-N, AceMath-7B-RM	$N=10$	1.040e15	24.5	0.0
PRM rerank, Qwen2.5-Math-PRM-7B	$N=10$, last step	1.040e15	29.1	0.0
Recursive Refinement	$N=8, k=4, T=2$	1.048e15	17.9	5.4
CORAL	$K=8$	1.050e15	35.4	23.6

Table 6: Inference runtime and memory usage, Qwen3-1.7B, single H100.

Method	Mode	Out. budget	Prompt	Runtime				Memory		Accuracy	
				Consolidate	Decode	Total	Speed	Peak KV	Peak mem	Minority	None
		<i>tok</i>	<i>tok</i>	<i>s</i> ↓	<i>s</i> ↓	<i>s</i> ↓	<i>ms/tok</i> ↓	<i>GB</i> ↓	<i>GB</i> ↓	% ↑	% ↑
Prompt agg	CoT	8,192	6,182	–	25.867	25.867	3.158	1.649	15.742	17.2	4.0
SFT agg	CoT	8,192	6,182	–	25.867	25.867	3.158	1.649	15.742	10.2	3.2
Prompt agg	Thinking	16,384	6,178	–	55.038	55.038	3.359	2.588	15.761	25.8	13.0
RL agg	Thinking	16,384	6,178	–	55.034	55.034	3.359	2.588	15.742	22.0	3.8
CORAL	CoT	8,192	156	1.946	23.291	25.238	2.843	0.957	21.962	35.4	23.6

B Ablation Studies

B.1 Joint Bi-Level Optimization is Necessary

We verify that CORAL’s improvement arises from its bi-level consolidation mechanism itself, rather than simply from the frontier-model supervision used in its outer loop. SFT Aggregation in CoT mode is trained with the same questions, trajectory groups, and frontier-generated target solutions as CORAL-SFT, so both methods receive identical supervised examples and expert supervision. Table 7 compares the two methods on the 10 benchmarks from Table 1: CORAL-SFT consistently and substantially outperforms direct supervised aggregation.

CORAL uses bi-level optimization with two nested loops, where the inner loop is optimized by the outer loop. We isolate each loop’s contribution in turn.

Without Inner Loop. If the inner loop is removed, training reduces to either (1) SFT Aggregation, already shown above, or (2) SFT on math problems with a single CoT trace from the frontier teacher models (same frontier supervision as CORAL), where sampled trajectories are ignored. Table 8 reports this scenario for the *None* tier, both with and without aggregation at inference time. CORAL outperforms both inference settings under this SFT-trained baseline, with strong performance gains.

Without Outer Loop. If the outer loop is removed, meta-learning is removed, and the inner-loop-only ablation applies the inner-loop update at test time from the default, untrained LoRA initialization — this is plain test-time training (TTT) without meta-learning. Table 9 shows that this ablation performs much worse than the complete CORAL under the same evaluation setting.

Together, these two ablations show that direct outer-loop supervision (i.e., SFT) is insufficient, while inner-loop updates without meta-learning are also ineffective. CORAL’s gains arise from optimizing the two components jointly in a bi-level optimization framework, so that the outer loop learns how the inner-loop adaptation should consolidate trajectories.

B.2 Isolating the Source of Improvement

We rule out three alternative explanations for CORAL’s improvement that do not require genuine cross-trajectory consolidation: repeated exposure to the problem, generic task-distribution activation, and gains available from the learned pre-adaptation initialization alone. Table 10 reports matched

Table 7: Same frontier supervision, different consolidation mechanism: SFT Aggregation vs. CORAL-SFT on the 10 benchmarks from Table 1.

Model	Trajc Tier	SFT Agg (CoT)	CORAL-SFT	Gain
Qwen3-1.7B	None	3.2	23.6	+20.4
Qwen3-1.7B	Minority	10.2	35.4	+25.2
Qwen3-4B	None	7.7	40.8	+33.1
Qwen3-4B	Minority	15.4	57.6	+42.2

Table 8: Outer-loop-only ablation: removing the inner loop reduces training to SFT on a single frontier-teacher CoT trace (sampled trajectories ignored), evaluated with and without aggregation at inference; Qwen3, *None* tier.

	AIME26	Apex25	ArxivMath	HMMT2602
<i>No aggregation at inference</i>				
SFT-1.7B	3.3	0	5.6	3.0
SFT-4B	14.2	0	11.1	6.1
<i>With aggregation at inference</i>				
SFT-1.7B	0.8	0	0.7	1.5
SFT-4B	3.3	0	2.4	3.0
<i>CORAL (test-time consolidation)</i>				
CORAL-1.7B	26.7	16.7	23.6	18.2
CORAL-4B	39.1	33.3	37.9	48.5

Table 9: Inner-loop-only ablation (test-time updates from an untrained initialization, no outer-loop meta-learning) vs. full CORAL.

Backbone	Setting	AIME26	Apex25	ArxivMath	HMMT2602
Qwen3-1.7B (None)	Only inner	5.0	0	10.8	3.8
	Inner + outer	26.7	16.7	23.6	18.2
Qwen3-1.7B (Minority)	Only inner	6.7	4.2	9.0	4.6
	Inner + outer	30.0	25.0	34.7	27.3
Qwen3-4B (None)	Only inner	14.2	0	12.2	13.6
	Inner + outer	39.1	33.3	37.9	48.5
Qwen3-4B (Minority)	Only inner	15.0	0	14.9	9.1
	Inner + outer	47.6	41.6	42.9	57.7

controls for each. We additionally isolate cross-trajectory interaction directly with a **Single-Trace $\times$ 8** baseline, where we independently adapt on a single trajectory in the group, obtaining 8 different LoRAs, each generating one final solution, and apply majority voting to determine the final answer. This condition uses all eight trajectories and similar total adaptation compute as full CORAL, and produces eight final solutions. However, information cannot be shared across trajectories before generating the solutions.

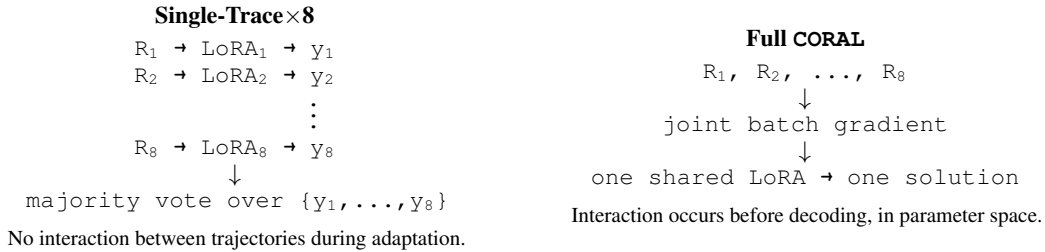


Figure 8: Where information can interact under Single-Trace $\times$ 8 vs. full CORAL.

Single-Trace $\times$ 8 performs much worse than CORAL, showing that independently encoding each trajectory and voting over the generated answers cannot reproduce CORAL’s improvement. The other controls also perform far below CORAL, showing that passive question repetition and task-distribution activation do not contribute to CORAL’s improvement. (To make the $8\times$ Single-Trace baseline a fair comparison to CORAL, we additionally sample 8 CoTs from the adapted model after the inner loop and perform a majority vote of these, which yields a $\sim 1\%$ average additional improvement.)

CORAL requires information from multiple target-specific trajectories to be jointly consolidated in the same adapted LoRA before it generates the final solution. We accordingly scope the weak-to-strong generalization claim to: **CORAL can recover from unsuccessful trajectories and arrive at an improved solution through aggregation as test-time learning.**

Table 10: Controls isolating the source of CORAL’s improvement.

Condition	Inner-loop input	Avg. acc.	Δ pre-adapt.	Gap to CORAL	Isolates
Pre-adaptation	No inner-loop update	7.5%	–	–14.5	Initialization alone
Passive repetition	8 copies of the target question	5.4%	–2.1	–16.6	Repeated exposure
Task activation	Trajectory groups from other problems, same task distribution	4.0%	–3.5	–18.0	Task-distribution activation w/o target-specific info
Single-Trace $\times$ 8	One trajectory per LoRA, 8 LoRAs, majority vote	11.7%	+4.2	–10.3	Absence of cross-trajectory interaction
Full CORAL	All 8 target trajectories, jointly	22.0%	+14.5	–	–

Table 11: Same checkpoint, with vs. without test-time inner-loop adaptation (Qwen3-1.7B, *None*).

		AIME26	Apex25	ArxivMath	HMMT2602	GPQA
CORAL-SFT	Before test-time update	16.7	0	2.8	6.1	6.6
	After test-time update	26.7	16.7	23.6	18.2	22.7
	Gain from adaptation	+10.0	+16.7	+20.8	+12.1	+16.1
CORAL-RL	Before test-time update	16.7	0	4.2	9.1	19.7
	After test-time update	23.5	16.7	26.0	21.7	40.0
	Gain from adaptation	+6.8	+16.7	+21.8	+12.6	+20.3

B.3 Role of Reinforcement Learning in Consolidation

Because RL jointly optimizes the LoRA initialization *and* the model’s reasoning after inner-loop updates, the effects of initialization and consolidation are coupled in the post-adaptation measurement. To directly separate performance available in the initialization from performance obtained after trajectory consolidation, we evaluate the same RL-enhanced CORAL model with and without consolidation at inference time (Table 11).

The only difference between the two rows within each checkpoint is whether the inference includes the inner-loop adaptation. The inner-loop adaptation clearly improves every benchmark, so the final CORAL model does not obtain its performance from the initialization alone; its test-time updates contribute a substantial improvement. CORAL-RL is initialized from the CORAL-SFT policy before GRPO training, following prior art, so it does inherit frontier supervision through this initialization. To isolate RL’s own contribution, we additionally evaluate a compute- and initialization-matched RL Aggregation baseline that initializes from SFT Aggregation and applies the same GRPO training, verifiable reward, rollout budget, and RL update budget used for CORAL-RL (Table 12).

This comparison controls for both frontier-supervised initialization and RL training; CORAL still outperforms this baseline under the matched SFT-to-RL learning path. We do not interpret this as showing that RL primarily improves the consolidation process (though this is benchmark-dependent); however, the adaptation gain under the same checkpoint shows that the inner loop remains important. RL primarily strengthens the initialization, but the resulting model still relies substantially on trajectory-conditioned test-time adaptation: *RL improves the final adapted policy’s ability to reason over consolidated trajectories, while its current training gains arise mainly from a stronger initialization for consolidation.*

Table 12: Matched SFT→GRPO comparison, Qwen3-1.7B, *None* tier, avg. over 4 benchmarks.

Method	AIME26	ArXivMath	Apex25	HMMT-2602	Avg
RL Aggregation: base → GRPO (base-initialized)	6.7	2.8	0.0	0.4	2.48
RL Aggregation: SFT → GRPO (matched)	10.0	1.2	0.0	9.1	5.08
CORAL-SFT → GRPO (CORAL-RL)	23.5	26.0	16.7	21.7	21.98

C Consolidation Dataset: Construction, Quality Control, and Verification

C.1 Evaluation Benchmark and Trajectory Preparation

Benchmark Coverage and Question Pool

We assemble a pool of 1,645 unique questions drawn from ten challenging reasoning benchmarks spanning competition mathematics and scientific reasoning. We detail the full pipeline to construct the reasoning consolidation dataset used in our experiments. Table 13 provides a complete breakdown of each benchmark, its domain, and the number of questions it contributes. Together, these benchmarks cover a broad difficulty spectrum: from competition-level problems designed for elite high-school contestants (AIME, HMMT, CMIMC) and olympiad-level challenges (IMO-Bench) to research-level mathematics (ArXivMath), PhD-level scientific reasoning across physics, chemistry, and biology (GPQA Diamond), and multi-domain STEM theorem application (TheoremQA). Notably, the difficulty of these benchmarks substantially exceeds the reach of smaller models, posing a genuine challenge even for capable reasoning models.

Table 13: Summary of the ten reasoning benchmarks. Each question in the dataset is used to generate 40 candidate trajectories (5 source models $\times$ 8 samples), from which three quality-stratified trajectory groups are constructed.

Benchmark	Domain	# Questions
<i>Math Reasoning</i>		
AIME 2025	Competition Math	30
AIME 2026	Competition Math	30
Apex 2025	Competition Math	12
ArXiv Math	Research Math	72
CMIMC 2025	Competition Math	40
HMMT Feb 2026	Competition Math	33
HMMT Nov 2025	Competition Math	30
IMO-Bench	Olympiad Math	400
<i>STEM Reasoning</i>		
GPQA Diamond	Scientific Reasoning	198
TheoremQA	STEM Theorem Reasoning	800
Total		1,645

Source Models and Trajectory Generation For each question in the pool, we sample 8 independent solution attempts from each of five source models, yielding 40 candidate trajectories per question. The five source models span a range of scales and capabilities. **Qwen2.5-1.5B-Instruct**, **Qwen3-1.7B**, and **Qwen3-4B** represent lightweight to mid-range open models. **Qwen3-30B-A3B** is a sparse mixture-of-experts model with 30B total but only 3B active parameters per token, offering strong reasoning at moderate inference cost. **Gemini 2.0 Flash** is a capable closed-source model serving as the strongest single contributor. The first four open-source models operate in extended thinking mode and produce very long internal reasoning traces before writing a final answer. While Gemini 2.0 Flash is a closed-source model that outputs only a brief CoT solution without an explicit reasoning trace.

CoT Extraction from Thinking Outputs For the four open-source thinking models, each raw generation contains a lengthy internal reasoning trace in `<think>...</think>` tags, which can

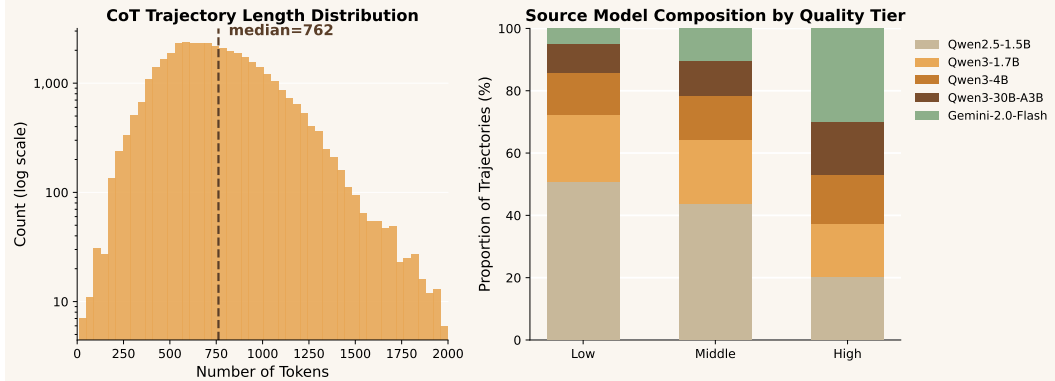


Figure 9: **Left:** Distribution of extracted CoT lengths over reasoning trajectory collection. **Right:** Source model composition by quality tier, where weaker models dominate the *None* tier while stronger models contribute more in the *Majority* tier.

exceed tens of thousands of tokens and is therefore too long to include directly in aggregation prompts. We extract only the brief final solution appearing after the thinking block as the CoT trajectory. For Gemini 2.0 Flash the full output is used directly. As shown in Figure 9 (left), the extracted CoTs have a median length of 762 tokens (25th–75th percentile: 559–927), confirming they are compact enough for multi-trajectory aggregation contexts.

Quality-Based Stratification For each question, we sample 8 trajectories from the 40 candidates and assign them to one of three quality tiers based on how many of the 8 are correct:

- *None* (0/8 correct): all selected trajectories are incorrect, representing the hardest aggregation scenario where the model must extract useful reasoning steps from entirely wrong solutions.
- *Minority* (0–3/8 correct): a mix of correct and incorrect trajectories, reflecting the most common real-world setting where the aggregator must identify and leverage correct solutions among a noisy mixture.
- *Majority* (4–7/8 correct): the majority of selected trajectories are correct, an extreme case testing whether the model can reliably synthesize a consistent final answer.

Two boundary cases arise in practice: some questions are so easy that all 40 candidates are correct ($\text{pass@40} = 100\%$), and others are so hard that all 40 are wrong ($\text{pass@40} = 0\%$). For these questions, we use frontier models (GPT-5.3 and Claude 4.6) to manually correct the imbalance with access to the ground-truth answer. In the all-correct case, a subset of trajectories is rewritten to introduce plausible reasoning errors that lead to a wrong final answer. In the all-wrong case, a subset of trajectories is corrected to produce a valid solution. This ensures that every question participates in all three quality tiers, yielding $1,645 \times 3 = 4,935$ trajectory groups in total.

Source Model Composition across Quality Tiers As shown in Figure 9 (right), source model composition shifts systematically across quality tiers. In the *None* tier, 51% of trajectories come from Qwen2.5-1.5B, the weakest model, because it fails most often. As quality increases, stronger models contribute more: Gemini 2.0 Flash rises from 5% (*None*) to 30% (*Majority*), and Qwen3-30B-A3B from 9% to 17%, while Qwen2.5-1.5B’s share falls to 20%. This emergent shift confirms that model capability and trajectory quality are tightly coupled.

C.2 OpenParallelThinking Training Corpus Construction

This section details the construction pipeline for *OpenParallelThinking* summarized in Section 3.

Source Diversity We pool problems from OmniMath [22], DeepScale [38], NuminaMath [32], and OpenMathInstruct [52], yielding 19,615 unique problems after hash-based deduplication.

Trajectory Diversity We sample from four solvers spanning different scales, families, and training recipes: R1-7B [16], Nemotron3-14B [11], Qwen3-4B, and Qwen3-30B-A3B [65], since a homogeneous source would bias the aggregator toward a single reasoning style. We drop the extended reasoning segment of each trajectory with probability 0.5³, exposing the aggregator to a realistic mix of detailed and compact reasoning. Trajectories are then randomly grouped per question under a 16K-token budget, so that group size varies naturally with length (Figure 2, left).

Balanced Difficulty We filter trajectory groups by correctness ratio (Figure 2, center) to provide more training examples on reasoning aggregation over groups where the majority are incorrect trajectories, which provides stronger learning signals without the majority correct answer shortcut.

Final Dataset The final release contains 12,810 questions and 63,507 trajectory groups. We sample 8,000 instances for RL training and reserve the rest for SFT.

³For thinking models, the reasoning segment corresponds to the content between two special thinking tokens, most commonly `<think>` and `</think>`.

C.3 CoT-Only vs. Full Thinking + CoT Consolidation

Our main experiments consolidate the extracted final-answer CoT segment of each trajectory (Appendix C.1) rather than the complete thinking trace. Table 14 compares consolidating CoT-only trajectories against full thinking+CoT trajectories (Qwen3-1.7B).

We observe that when consolidating over a group of complete trajectories (thinking + CoT), the benefit is mixed: some benchmarks improve while others are flat or slightly worse. Our hypothesis is that (1) the CoT part already contains sufficient useful information, and (2) a single full thinking + CoT trajectory is $\sim 15.7K$ tokens on average, so a group of 8 averages $\sim 125.6K$ tokens, which likely introduces a length-generalization difficulty of its own. We therefore retain CoT-only consolidation as the main configuration and leave full-trace consolidation, and the length-generalization question it raises, to future work.

Table 14: CoT-only vs. full thinking+CoT trajectory consolidation.

		AIME26	Apex25	ArxivMath	HMMT2602
None	CoT	26.7	16.7	23.6	18.2
	Full	26.7	16.7	26.4	21.2
Minority	CoT	30.0	25.0	34.7	27.3
	Full	33.3	25.0	36.1	24.2

C.4 Verifying Genuine Trajectory Consolidation

A natural concern is whether CORAL’s inner loop learns to memorize input trajectories rather than consolidate useful sub-reasoning, and whether its gains on the *None* tier are an artifact of the frontier-model-rewritten trajectories used to balance that tier (Appendix C.1) rather than genuine consolidation of naturally occurring failures. We address both concerns with a construction-based argument and a new LLM-as-judge analysis.

Ruling out memorization by construction. When all 8 trajectories in the *None* tier are naturally unsuccessful trajectories sampled from source models, each ending in a wrong answer, there are no correct candidates to memorize and recall. Averaged over the ten benchmarks, CORAL reaches 23.58 vs. 13.02 for the strongest baseline (Qwen3-1.7B) and 40.83 vs. 19.63 (Qwen3-4B). For CORAL to output a correct answer in this setting, it must not have only memorized trajectories leading to the correct answer. Figure 4 also shows that CORAL generally improves as more incorrect trajectories are added, while baselines stagnate or degrade. Thus, CORAL does not depend on simply memorizing and copying the input trajectories’ answers.

Ruling out trajectory-editing artifacts. Edited trajectories are a negligible presence in the *None* setting. Among all 1,645 evaluation questions, only two (one from GPQA; one from TheoremQA) contain any rewritten trajectory in the *None* setting; the remaining eight benchmarks contain none. Counting these two questions as incorrect decreases the GPQA and TheoremQA scores by less than 0.5 percentage points and leaves every other benchmark score unchanged. The weak-to-strong gains in Table 1 are therefore primarily driven by naturally occurring model failures, not edited rewriting.

Judge Setup We use three frontier LLM judges, Fable 5, Opus 5, and GPT-5.6-Sol, each run at extra-high reasoning effort. Every sampled solution is independently labeled by all three judges, and the final label is the majority vote across the three.

Synthesis and Artifact-Exploitation Judging (*None*) We sample 100 CORAL (Qwen3-1.7B) solutions on *None*-tier problems, where every one of the eight input trajectories reaches an incorrect answer. For each solution, the judges are shown the problem, the eight input trajectories, and CORAL’s final solution, and are asked two binary questions: (1) whether the final solution derives an improved reasoning trajectory from the incorrect inputs (*synthesis*) rather than simply copying one input’s reasoning verbatim, and (2) whether the solution exploits stylistic or distributional cues rather than demonstrating mathematical understanding (*artifact exploitation*). 98/100 solutions are labeled as synthesis, and 97/100 show no evidence of artifact exploitation. We additionally audit the two *None*-tier questions containing an edited trajectory (Appendix C.1) specifically for exploitable artifacts; the judges find that the edits introduce mathematically meaningful errors without creating clear exploitable cues.

Table 15: Prompt template for the LLM-as-judge protocol.

Component	Content
Judge Prompt	You are evaluating whether a model’s final solution to a problem reflects {synthesis vs. artifact exploitation consolidation vs. memorization}. Problem: {problem} Input trajectories: {trajectory₀}, ..., {trajectory₇} Final solution: {solution} Question: {task-specific question} Respond with a single label from {label set} and a one-sentence justification.

Memorization Judging (*Minority*) We separately sample 100 CORAL (Qwen3-1.7B) solutions on *Minority*-tier problems, where a subset of the eight input trajectories is already correct. Judges are shown the same problem, trajectory, and solution triple, and asked to label the final solution as *memorization* (near-verbatim reproduction of one input trajectory’s reasoning and answer) or *consolidation* (reasoning synthesized or corrected using information across multiple inputs). 1/100 solutions is labeled memorization.

Judge Prompt Template Table 15 shows the prompt template shared across the three judging tasks, with the task-specific question and label set substituted for each protocol above.⁴

Taken together, these results show clear evidence that CORAL consolidates multiple naturally sampled unsuccessful trajectories into a stronger, improved solution, rather than memorizing correct answers or exploiting artifacts introduced by trajectory editing.

D Training and Inference Details

Hardware Information All experiments are conducted on a single NVIDIA H100 GPU with 93 GB of HBM on an internal Slurm-managed research cluster. Each job is assigned 64 CPU cores and 64 GB of system RAM with exclusive GPU access throughout training.

Baseline Training Hyperparameters The SFT aggregation model is obtained by full-parameter fine-tuning using LLaMA-Factory [71]. We adopt the no-thinking chat template with a context window of 20,576 tokens. Training runs for 1 epoch with a learning rate of 1×10^{-5} , a cosine schedule with 10% linear warmup, and an effective global batch size of 32. All training is conducted in bfloat16 precision. For RL aggregation, we implement GRPO [45] under the Verl [46] framework, with thinking mode enabled. We use a learning rate of 3×10^{-6} , a global batch size of 64 prompts, and $n = 5$ rollouts per prompt. The maximum prompt and response lengths are set to 8,192 and 16,384 tokens, respectively.

Our Method Training Hyperparameters We train CORAL for up to 1 epoch with early stopping based on validation loss, using PyTorch Lightning⁵ modified to support meta-gradients. The outer loop optimizer is AdamW [37] with a learning rate of 1×10^{-5} , weight decay of 0.01, and a cosine schedule with 3% warmup steps. The consolidation module is parameterized as a LoRA adapter with rank 256 and $\alpha = 128$, applied to all weight matrices of the model. For inner-loop adaptation, we reuse the encoding scheme and hyperparameters from that prior work, including its Truncated Gradient Unrolling (TGU) scheme [42]: the first $S - T$ inner steps are executed without graph retention and only the final T steps are differentiated, reducing memory from $O(S|\phi|)$ to $O(T|\phi|)$.

For the RL-enhanced variant, we introduce GRPO as the outer-loop objective on top of the existing PyTorch Lightning meta-training framework, without adopting dedicated RL infrastructure such as vLLM or VERL. The outer loop learning rate is reduced to 1×10^{-6} with weight decay 0.003. Rollouts are capped at 2,048 new tokens. To further reduce peak memory from higher-order gradient

⁴More detailed instructions, including definitions of artifact exploitation, memorization, and consolidation, will be released with the code.

⁵<https://lightning.ai/docs/pytorch/stable/>

Table 16: Prompt templates for model-based aggregation baselines and CORAL.

Method	Prompt Template
Prompt Aggregation SFT Aggregation RL Aggregation	Given the following problem: {problem} and these solution attempts: Solution attempt 0: {trajectory₀} Solution attempt 1: {trajectory₁} ... (more attempts) Carefully review the provided solutions, using them as starting points, correcting mistakes, filling in gaps, and/or combining useful ideas, to produce a final, comprehensive, and correct solution to the problem. Analyze step by step and put your final answer in the format \boxed{ }
CORAL (Inner Loop)	record₀: {trajectory₀} record₁: {trajectory₁} ... (one sequence per trajectory, K steps total)
CORAL (Outer Loop)	record₀ to record_{$K-1$} are the information you memorized. Question: {problem} Please solve this problem step by step by reasoning from your memory and put your final answer in the format \boxed{ }. \nAnswer:

computation, we set `unroll_start=3` with $n_{\text{inner}} = 4$, retaining only the final inner step in the gradient graph.

Evaluation Hyperparameters All methods use greedy decoding (temperature = 0), with the end-of-sequence token taken from the model tokenizer’s default. The maximum generation length is 8,192 tokens in CoT mode and 16,384 tokens in thinking mode. Baseline inference is accelerated with vLLM [29], while CORAL generates answers through the Hugging Face Transformers pipeline⁶.

E Prompts

The three model-based aggregation baselines (**Prompt Aggregation**, **SFT Aggregation**, and **RL Aggregation**) share an identical prompt template in training and inference, differing only in the underlying model checkpoint. Prompt Aggregation uses the base model directly, while SFT and RL Aggregation use models further fine-tuned via supervised fine-tuning and reinforcement learning, respectively. The shared prompt concatenates all K solution attempts with indexed headers and instructs the model to synthesize a final correct answer by reviewing them. CORAL uses two distinct prompts. The inner-loop prompt encodes each reasoning trajectory as a named memory record (`recordi`) using a causal language modeling objective, compressing the trajectory set into the LoRA adapter weights via gradient updates. The outer-loop prompt then instructs the adapted model to leverage memorized records to answer questions. Table 16 shows the detailed prompt templates.

F Limitations

Memory Cost of Meta-RL Training. Meta-learning requires backpropagating through the inner optimization loop, which increases memory cost compared to standard fine-tuning. Replacing the SFT outer loss with GRPO further amplifies peak memory usage due to rollout generation and higher-order gradient computation. LoRA substantially reduces this overhead, while we believe further reductions through gradient checkpointing or more aggressive truncation schemes would broaden accessibility to researchers with tighter hardware budgets and enable scaling to larger backbone models.

Scope of Reasoning Domains. Our experiments cover mathematical and STEM reasoning, where trajectory quality can be easily and completely assessed through verifiable final answers. Extending CORAL to agentic settings, where reasoning involves tool use, multi-step environment interaction, and open-ended planning, introduces new challenges in defining trajectory quality and designing appropriate reward signals. These are natural and compelling directions for future work, and the meta-learning formulation of CORAL is in principle compatible with such settings.

⁶https://huggingface.co/docs/transformers/v4.43.4/en/main_classes/pipelines